Maximum-Flow Network Interdiction via QUBO Solvers

Problem summary, derivations, and research notes — Network-Interdiction project

What this is. Part I summarizes the problem the repository attacks, the three reformulations it uses, what the code actually implements and the experimental protocol behind the group's D-Wave runs, the derivations that say when the QUBO's ground state is the right answer (with proofs, and a check against the sample benchmark), the D-Wave results on three graph families, and the surrounding literature. Part II holds the exploratory threads — the original brainstorm on convex relaxations versus SA/QA, two concrete warm-start methods from the literature, and the divide-and-conquer / subgradient work in progress — and ends by tying each thread to the specific result in Part I it depends on, which yields an ordering for the open issues, and by mapping the CQT Year-4 proposal's commitments onto those issues (Section 10). Sources are the `Network-Interdiction` repository, the group's slide deck `S-T_NIP.pptx` (16 slides; formulation, penalty argument, protocol, results, WIP), and the papers cited inline.

Contents

1 The problem **2**

2 Reformulations **3**

2.1 Single-level MILP by dualizing the inner max flow (Wood 1993) 3

2.2 What the duals are 3

2.3 Lagrangian relaxation of the budget 4

2.4 Penalized QUBO (`nip_to_qubo`) 4

3 What the repository implements **5**

4 Derivations and results **6**

4.1 Results 8

5 Existing work **9**

6 Convex relaxations vs. SA/QA heuristics — the original brainstorm **10**

7 Two concrete warm-start recipes from the literature — and whether we can implement them **12**

8 Divide and conquer, and subgradient tuning of λ — work in progress from the slides **15**

9 Tying the threads to the results **17**

10 The CQT Year-4 proposal: commitments and the issues that carry them **19**

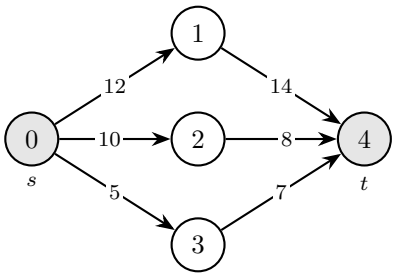
Part I — Problem summary

1 The problem

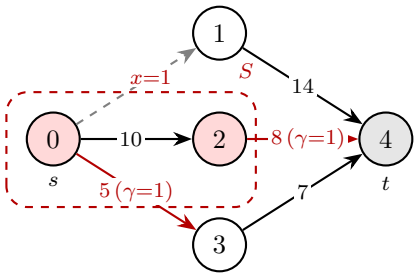
Maximum-flow network interdiction (MFNIP). Given a directed graph $G = (V, A)$ with source s, sink t, integer arc capacities $u_a > 0$, and an interdiction budget B, an attacker removes at most B arcs so as to minimize the maximum s–t flow that survives:

$$\min_{x\in\{0,1\}^A,\ \sum_a x_a \le B} \ \max\text{-flow}\big(G \setminus \{a : x_a = 1\}\big). \tag{1}$$

This is a bilevel (min–max) problem; Figure 1 shows it on the smallest benchmark instance. The repository's instances use unit interdiction cost per arc (so the budget is a cardinality), and integer capacities read from `testcases/*.flow`; the planar family comes from LEDA-generated maximal planar graphs with random arc orientations and random capacities (`leda_to_flow_file`).



(a) `tiny_b`: three s–t paths with bottlenecks 12, 8, 5; max flow $f(0) = 25$



(b) $B = 1$, interdict $(0, 1)$: min cut $S = \{0, 2\}$ (shaded nodes have $\pi = 1$) pays $8 + 5$: $f(1) = 13$

Figure 1: The problem on the smallest three-path benchmark instance. (a) The attacker sees the network and its capacities; the max flow is the sum of the three path bottlenecks. (b) With budget $B = 1$ the best single arc to remove is $(0, 1)$, the bottleneck of the widest path; the surviving max flow is 13, certified by the cut $S = \{0, 2\}$ whose crossing arcs $(2, 4)$ and $(0, 3)$ carry $8 + 5$. In the MILP (2) (whose π, γ are the max-flow LP's dual variables, Section 2.2), $x_a = 1$ marks the removed arc, $\pi_i = 1$ marks the source side S, and $\gamma_a = 1$ marks the crossing arcs. The removed arc $(0, 1)$ also crosses the cut (so the constraint $\pi_0 - \pi_1 \le \gamma_{(0,1)}$ forces $\gamma_{(0,1)} = 1$), but its capacity is not paid for: that is the bilinear term $u_a(1 - x_a)\gamma_a$ vanishing at $x_a = 1$. Node 1 sits on the sink side because putting it in S would make $(1, 4)$ cross at cost 14.

Complexity. MFNIP is NP-hard in general (Wood 1993; Phillips 1993). Two facts worth keeping

in view because they shape what "hard instance" can mean for issue #1:

- Phillips (1993) gives a pseudo-polynomial algorithm on *planar* graphs via the planar dual. The repository's benchmark family is planar. That does not make them easy for a general-purpose MILP solver, but it does mean "hard for Gurobi" on this family is a statement about the monolithic formulation, not about intrinsic hardness — worth confirming against Phillips' construction before building the benchmark set on planar instances alone.
- The LP relaxation of the standard single-level formulation below has a large integrality gap (Altner, Ergun & Uhan 2010, who also give valid inequalities), so the root bound is weak; Chestnut & Zenklusen (2017) give a 2-approximation and matching hardness-of-approximation results. The weak root bound is the specific reason branch-and-bound trees on this problem grow the way they do.

2 Reformulations

2.1 Single-level MILP by dualizing the inner max flow (Wood 1993)

Fix x. The surviving max flow equals the minimum s–t cut in G with capacities $u_a(1-x_a)$, and the min-cut LP

$$\min_{\gamma\ge 0,\ \pi} \sum_{a=(i,j)} u_a(1-x_a)\,\gamma_a \quad \text{s.t.} \quad \gamma_a \ge \pi_i - \pi_j \ \ \forall a, \qquad \pi_s - \pi_t \ge 1$$

has a totally unimodular constraint matrix, so it has an integral optimum with $\pi \in \{0,1\}^V$ marking the source side of the cut and $\gamma_a \in \{0,1\}$ marking cut arcs. Substituting this for the inner problem turns (1) into a single-level 0–1 program:

$$\min_{x,\gamma,\pi\in\{0,1\}} \sum_a u_a(1-x_a)\gamma_a \quad \text{s.t.} \quad \pi_i - \pi_j - \gamma_a \le 0 \ \ \forall a=(i,j), \qquad \pi_t - \pi_s \le -1, \qquad \sum_a x_a \le B. \tag{2}$$

This is exactly `solve_nip_gurobi`. The objective is bilinear in (x,γ); Wood linearizes it with $\gamma_a \ge \pi_i - \pi_j - x_a$ (an interdicted arc need not be paid for even if it crosses the cut), whereas the repository keeps the product and lets Gurobi linearize binary products internally. Both are exact.

2.2 What the duals are

The π and γ variables are not modelling conveniences: they are the dual variables of the max-flow linear program, and that is the reason (2) is exact. Written out, for a fixed interdiction x the inner problem is

$$\begin{aligned}
\text{(max-flow)} \quad & \max_{\phi\ge 0} \sum_{a\in\delta^+(s)} \phi_a \\
\text{s.t.} \quad & \sum_{a\in\delta^+(i)} \phi_a - \sum_{a\in\delta^-(i)} \phi_a = 0 && \forall i \in V\setminus\{s,t\} \quad \text{(dual variable } \pi_i\text{)} \\
& \phi_a \le u_a(1-x_a) && \forall a \in A \quad \text{(dual variable } \gamma_a\text{)}
\end{aligned}$$

with ϕ_a the flow on arc a. Attaching a multiplier to each constraint — a node potential π_i to conservation at i, a price $\gamma_a \geq 0$ to the capacity of a — and taking the LP dual gives

$$\text{(min-cut)} \qquad \min_{\gamma \geq 0,\ \pi} \ \sum_a u_a(1 - x_a)\,\gamma_a \quad \text{s.t.} \quad \gamma_a \geq \pi_i - \pi_j \ \ \forall a = (i,j), \qquad \pi_s - \pi_t \geq 1,$$

which is the LP in the previous subsection. Strong LP duality says its optimal value *equals* the max flow, for every x; that is the max-flow/min-cut theorem, obtained here as a corollary of duality rather than as a separate combinatorial fact.

Reading the duals. Fix $\pi_s = 1, \pi_t = 0$ (only the difference matters). An optimal dual can always be taken with $\pi_i \in \{0,1\}$, so π is the indicator of a set $S \ni s$, $t \notin S$: the source side of a cut. The constraint $\gamma_a \geq \pi_i - \pi_j$ then forces $\gamma_a = 1$ on exactly the arcs that leave S, and $\gamma_a = 0$ is optimal everywhere else, so the objective is the capacity of the cut $(S, V \setminus S)$. Figure 1(b) is this reading drawn out. Complementary slackness supplies the economic interpretation: $\gamma_a > 0$ only on arcs that are saturated ($\phi_a = u_a(1 - x_a)$), and flow is positive only on arcs with $\gamma_a = \pi_i - \pi_j$ — the arcs on which the potential drop is exactly the price. Node potentials that increase by one across the cut and are flat elsewhere are the certificate that no more flow can pass.

Why integrality is free. The dual constraint matrix is the transpose of a network matrix, hence totally unimodular; every vertex of the dual polytope is 0/1. That is what lets (2) declare π, γ binary without changing the optimum, and it is the observation, made in Part II, that the only fractionality in the LP relaxation of (2) comes from the budget row.

A second, different dual. The multiplier λ in (3) is a *Lagrangian* dual variable for the budget constraint of the outer problem, not an LP dual: the inner variables stay integer, so only weak duality holds, $L(\lambda) \leq$ opt for all $\lambda \geq 0$, and the best bound $\max_\lambda L(\lambda)$ can sit strictly below the optimum. Proposition 3 is precisely the statement of when that gap is zero. The naming in the code follows this split: the "dual penalty" β enforces the min-cut (LP-dual) constraints inside the QUBO, and the "cardinality penalty" λ *is* the Lagrangian dual variable, used as a fixed penalty coefficient rather than optimized.

2.3 Lagrangian relaxation of the budget

Move the budget into the objective with multiplier $\lambda \geq 0$:

$$L(\lambda) = \min_{x,\gamma,\pi} \ \sum_a u_a(1 - x_a)\gamma_a + \lambda\Big(\sum_a x_a - B\Big) \quad \text{s.t. the cut constraints of (2).} \tag{3}$$

This is what `solve_nip_gurobi_relaxation` solves repeatedly, bisecting on λ until the minimizer uses at most B arcs. It is a *Lagrangian* relaxation, not an LP relaxation: the variables stay binary, and $L(\lambda) \leq$ the optimum of (2) for every $\lambda \geq 0$ (weak duality). Section 4 says exactly when some λ recovers the optimum.

2.4 Penalized QUBO (`nip_to_qubo`)

The QUBO route keeps the Lagrangian treatment of the budget and additionally penalizes the cut constraints. Write $\pi_s \equiv 1$, $\pi_t \equiv 0$ (substituted, so the $\pi_t - \pi_s \leq -1$ constraint is enforced exactly

and those two qubits do not exist). The constraint $\pi_i - \pi_j - \gamma_a \le 0$ on binaries is violated in exactly one pattern, $(\pi_i, \pi_j, \gamma_a) = (1, 0, 0)$, so its natural penalty is the cubic indicator

$$\mathbb{1}[\text{violated}] = \pi_i(1-\pi_j)(1-\gamma_a) = \pi_i - \pi_i\pi_j - \pi_i\gamma_a + \pi_i\pi_j\gamma_a.$$

A QUBO cannot carry the cubic monomial, so `nip_to_qubo` introduces one auxiliary binary w_a per arc and uses

$$P_a(\pi_i, \pi_j, \gamma_a, w_a) = \pi_i - \pi_i\pi_j - \pi_i\gamma_a + w_a(\pi_i + \pi_j + \gamma_a - 1) + (\pi_i\pi_j + \pi_i\gamma_a + \pi_j\gamma_a) - (\pi_i + \pi_j + \gamma_a), \quad (4)$$

which simplifies to $P_a = \pi_j\gamma_a - \pi_j - \gamma_a + w_a(\pi_i + \pi_j + \gamma_a - 1)$. The full Hamiltonian, with coefficients (λ, β) passed as `(cardinality_penalty_coeff, dual_penalty_coeff)` and $\beta = \lambda + 1$ wherever the repository calls it, is

$$H(x, \gamma, \pi, w) = \sum_a u_a(1 - x_a)\gamma_a + \lambda\Big(\sum_a x_a - B\Big) + \beta \sum_a P_a, \quad (5)$$

over $n = 3|A| + |V| - 2$ binary variables. Note the cardinality term is *linear* in $\sum x_a$, not the squared $(\sum x_a - B)^2$ one usually sees — it is a Lagrangian term, and that single fact explains most of the penalty-tuning behaviour observed in the repository (Section 4).

3 What the repository implements

- **Exact:** (2) in Gurobi (`solve_nip_gurobi`); the penalized model (5) also in Gurobi as an exact check of a given (λ, β) (`gurobi_verify_parameters`); Lagrangian bisection (3) (`solve_nip_gurobi_relaxation`). All need a Gurobi license beyond trial size.
- **Heuristic:** build (5) with PyQUBO, sample it with simulated annealing (`dwave-neal`, 2500 reads) or a D-Wave Advantage QPU (`EmbeddingComposite`, 2500 reads, 50 μs anneal, followed by steepest descent), decode the 20 lowest-energy samples, evaluate each decoded x by an *actual* max-flow computation rather than by its energy, and repair each to exactly B arcs with a greedy add/remove pass (`improve_dwave_solution`). Reported: best surviving flow among budget-feasible samples, best after greedy repair, and best energy.
- **Unsupported:** the iterative λ-search variants that shelled out to a Julia QUBO builder (`nip_cubic.jl`, issue #5) raise `NotImplementedError`.
- **Benchmark pipeline (PR #13):** `run_benchmark.py` produces one CSV row per (instance, B, λ) with exact reference columns from Gurobi or, for tiny instances, from exhaustive search plus `dimod.ExactSolver`; `network_interdiction_plot.py` draws flow-reduction and energy ratios against the exact optimum.

Experimental protocol behind the D-Wave runs (slide 7 of `S-T_NIP.pptx`; the code paths above are its implementation):

- *Instances.* Three random families: maximal planar graphs (embedded with the linear-time algorithm of Stamm-Wilbrandt 1993; the `testcases/*.u` LEDA files), 2D k-nearest-neighbour graphs, and Erdős–Rényi graphs. Each undirected edge is oriented at random, then reverse arcs are added until the digraph is strongly connected; capacities are random integers. Budgets B from 4 to 12.

- *Penalty coefficients.* λ_1 (our λ) by binary search over $[\min_a u_a, \max_a u_a]$; λ_2 (our β) set to λ_1+1. Because QPU time was budgeted, the search is run with classical simulated annealing as the inner solver, and the QPU is queried once, at the final λ_1.
- *Sampling and decoding.* 2500 samples; the 20 with the lowest QUBO energy are decoded and their surviving flow computed exactly; the best is reported. Each of the 20 is also greedily post-processed to exactly B arcs: if over budget, remove the interdicted arcs with the least impact one at a time; if under, add the arc with the most impact one at a time (`improve_dwave_solution`).

4 Derivations and results

Everything below is about when the ground state of (5) is the optimum of (2). Let $f(k)$ denote the best surviving flow using at most k interdicted arcs (f is nonincreasing, $f(0)$ is the max flow), and $u_{\max} = \max_a u_a$.

Proposition 1 (The auxiliary variable quadratizes the cubic penalty exactly)**.** *For every* $(\pi_i, \pi_j, \gamma_a) \in \{0,1\}^3$,

$$\min_{w_a \in \{0,1\}} P_a(\pi_i, \pi_j, \gamma_a, w_a) = \pi_i(1-\pi_j)(1-\gamma_a) - 1,$$

and $P_a \geq \pi_i(1-\pi_j)(1-\gamma_a) - 1$ *for either value of* w_a*.*

Proof. Using the simplified form $P_a = \pi_j\gamma_a - \pi_j - \gamma_a + w_a(\pi_i + \pi_j + \gamma_a - 1)$: the w_a-free part equals -1 unless $(\pi_j, \gamma_a) = (0,0)$, where it is 0. The coefficient of w_a is $\pi_i + \pi_j + \gamma_a - 1 \in \{-1, 0, 1, 2\}$, so $\min_{w_a} w_a(\cdot) = -1$ when the coefficient is -1, i.e. when $(\pi_i, \pi_j, \gamma_a) = (0,0,0)$, and 0 otherwise. Checking the eight cases: the minimum is 0 at $(1,0,0)$ — the violating pattern — and -1 everywhere else, which is the claimed identity. The lower bound holds because a minimum over w_a is at most either value. □

Slide 6 of `S-T_NIP.pptx` states this quadratization as an equality, $\min a\,xyz = \min_w a\{w(x+y+z-1) + xy + yz + xz - (x+y+z)\}$; it holds up to the additive constant $-a$, which is harmless for the minimizer and is where the offset below comes from.

Two consequences. First, the ground-state energy of (5) carries a constant offset of $-\beta|A|$, which is why every `Opt_QUBO` in the sample CSV is negative; energies are comparable across λ only after removing it. Second, since w_a enters no other term, minimizing H over w is exact, and one may reason about H with $\beta \sum_a \mathbb{1}[\text{violated}_a]$ in place of $\beta \sum_a P_a$.

Proposition 2 (Dual penalty)**.** *If* $\beta > \lambda$*, every ground state of* H *violates no cut constraint. The same holds if* $\beta > u_{\max}$*, for every fixed* x *separately. In either case a ground state's energy is the true surviving max flow max-flow*$(G \setminus \{a : x_a = 1\})$ *plus* $\lambda(\sum_a x_a - B) - \beta|A|$*.*

Proof. Take any configuration with a violated arc $a = (i,j)$: $\pi_i = 1, \pi_j = 0, \gamma_a = 0$. A violation is an arc that leaves the cut side S without being paid for — an interdiction that dodges the λ charge. Make it honest: set $\gamma_a = 1$ and $x_a = 1$. The cut term $u_a(1-x_a)\gamma_a$ is 0 before and after, the β charge disappears, and the λ charge appears only if x_a was 0; the net change is at most $\lambda - \beta < 0$. (If x_a was already 1, setting $\gamma_a = 1$ alone saves β at no cost.) So under $\beta > \lambda$ every

violating configuration is strictly dominated and no ground state has one. The $u_{\max}$ variant repairs the violation without touching x: setting $\gamma_a = 1$ costs at most $u_a(1 - x_a) \le u_{\max}$ and saves β. A violation-free (γ, π) with $\pi_s = 1, \pi_t = 0$ is an s–t cut of cost $\sum_{a \in \text{cut}} u_a(1 - x_a)$; minimizing over cuts gives the min cut, i.e. the max flow on the surviving graph. □

The first condition is the one that matters: it is the argument on the "proof for penalty coefficients" slide of `S-T_NIP.pptx` ($\lambda_2 \ge \lambda_1$ in the slides' notation, strict here for uniqueness), and it is exactly why the repository's choice $\beta = \lambda + 1$ is safe — on every instance, independent of the capacities. On the sample rows β is below $u_{\max}$ (e.g. $\beta = 7$ against $u_{\max} = 10$ on `tiny_a`), so the second, capacity-based condition is not what is protecting those runs; the first is. The practical consequence is that the dual penalty is not a tuning problem at all: only λ is.

Proposition 3 (Valid window for the linear cardinality penalty)**.** *Assume β satisfies Proposition 2. The ground state of H interdicts exactly B arcs and attains $f(B)$ if and only if*

$$\max_{k>B} \frac{f(B) - f(k)}{k - B} \quad < \quad \lambda \quad < \quad \min_{k<B} \frac{f(k) - f(B)}{B - k}, \tag{6}$$

with non-strict inequalities giving ties. In particular a valid λ exists if and only if B is a vertex of the lower convex envelope of $k \mapsto f(k)$.

Proof. By Proposition 2, after minimizing over (γ, π, w) the energy of a configuration with k interdicted arcs is at least $f(k) + \lambda(k - B) - \beta|A|$, with equality for the best k-arc set. So the ground state minimizes $g(k) = f(k) + \lambda k$ over k, and $k = B$ is the unique minimizer exactly when $f(B) + \lambda B < f(k) + \lambda k$ for all $k \ne B$. Rearranging for $k < B$ and $k > B$ separately gives the two sides of (6). The set of k minimizing $f(k) + \lambda k$ for some $\lambda \ge 0$ is precisely the set of vertices of the lower convex envelope of f (a supporting line of slope $-\lambda$ touches f at k), which is the existence statement. □

The slides state a different range, $\lambda_1 \in (\min_a u_a, \max_a u_a)$, with an argument that adding or removing an interdicted arc changes the objective by u_a. That is the step that does not hold: interdicting an arc changes the surviving flow by its *marginal* effect on the min cut, which is at most u_a and is 0 for an arc on no minimum cut. So the slide range is neither necessary nor sufficient. It coincides with (6) on `tiny_a` and is a strict superset on the other sample instances — on `tiny_b` with $B = 1$ it admits $\lambda = 6$, for which $g(k) = f(k) + 6k$ takes the values $25, 19, 17, 18$ at $k = 0, 1, 2, 3$, so the ground state interdicts two arcs and overshoots the budget. The bisection the slides describe (and `solve_nip_gurobi_relaxation` implements) still lands inside the true window in practice, because it searches for the largest λ at which the minimizer respects the budget, which is the upper end of (6) whenever that window is non-empty; it just cannot tell when the window is empty.

This is the mechanism behind the "too-large λ makes the annealer interdict nothing" behaviour noted in the test suite and behind the narrow windows found when the sample CSV was generated: a large λ makes g increasing so $k = 0$ wins, a small λ makes k overshoot B, and if f is not convex at B no λ works at all — the standard Lagrangian duality gap for integer programs, showing up here as "no penalty coefficient exists." Table 1 checks (6) against the windows found empirically by sweeping λ with `dimod.ExactSolver` on the six sample rows.

instance	B	$f(0), f(1), f(2), f(3)$	window (6)	sweep hits (λ step 0.5)	λ used
`tiny_a`	1	15, 5, 0, 0	$(5, 10)$	$5.5, 6, \ldots, 10$	6.0
`tiny_b`	1	25, 13, 5, 0	$(8, 12)$	$8.5, 9, 9.5, \ldots$	9.0
`tiny_b`	2	25, 13, 5, 0	$(5, 8)$	$5.5, 6, 6.5, \ldots$	6.0
`tiny_c`	1	11, 4, 0, 0	$(4, 7)$	$4^*, 4.5, 5, \ldots$	4.5
`tiny_d`	1	22, 13, 6, 0	$(7, 9)$	$7.5, 8, 8.5$	8.0
`tiny_d`	2	22, 13, 6, 0	$(6, 7)$	$6^*, 6.5, 7^*$	6.5

Table 1: Predicted valid λ window versus the values at which the exact QUBO ground state was observed to decode to an optimal B-arc interdiction. Starred entries sit on a window boundary, where the ground state is degenerate and the solver happened to return the right member of the tie. Every observed hit lies in the predicted window and every predicted interior point that was tried was a hit.

4.1 Results

In the repository. The only committed results are the sample benchmark: on the six tiny rows, simulated annealing (2500 reads) reaches the exact ground-state energy and the exact optimal flow both before and after greedy repair, so every plotted ratio is 1.0. That demonstrates the pipeline, not performance — at 14–22 variables SA is effectively exhaustive.

D-Wave results in the slide deck. The runs the plotting script was written for exist as figures in `S-T_NIP.pptx` (slides 8–10), one per graph family, though the CSVs behind them were never checked in. Each figure reports, per instance and budget, the flow reduction achieved relative to the Gurobi optimum (1.0 = optimal) for raw QA and SA and for their greedy-repaired versions; a missing raw bar means none of the 20 best samples was budget-feasible. The x-axis label is the QUBO variable count $|V| + 3|A|$ (the planar sizes $914, 1389, 2339$ are the 50-, 75- and 125-node `testcases` instances). Table 2 is read off those figures, so values are approximate to about ± 0.01.

family	QUBO vars (B)	raw QA	raw SA	QA-greedy	SA-greedy
planar	914 (5, 6, 8, 10)	0.87–0.92	0.84–0.93	1.0	1.0
	1389 (5, 8, 10)	0.60–0.73	–, –, 0.82	1.0	1.0
	2339 (6, 10)	–, –	0.16, –	1.0	1.0
Erdős–Rényi	1124 (5, 6, 8)	0.89–0.95	0.91–0.93	1.0	1.0
	1438 (7, 8, 10)	0.78–0.97	0.89–1.0	1.0, 1.0, 0.99	1.0, 1.0, 0.99
	2370 (8, 10, 12)	0.70–0.86	0.85–0.95	1.0, 1.0, 0.97	1.0, 1.0, 0.97
2D k-NN	1070 (4, 5, 6)	0.37, –, 0.47	–, –, –	1.0	1.0
	1845 (6, 7, 8)	–, –, 0.86	0.81, 0.79, 0.86	0.95, 0.94, 0.93	0.95, 0.94, 0.93
	2398 (6, 7, 8)	0.52–0.90	0.66–0.90	0.90, 0.90, 0.94	0.90, 0.90, 0.94

Table 2: Flow reduction relative to the Gurobi optimum on the D-Wave Advantage QPU (QA) and simulated annealing (SA), from slides 8–10 of `S-T_NIP.pptx`. A dash means no budget-feasible sample among the 20 lowest-energy ones (the plots draw these as zero-height bars). Greedy columns are after the repair pass of Section 3.

Four things the table says, which the per-family slides do not say together:

1. *Raw annealer output degrades with size and is often infeasible.* At $\sim$ 900–1100 variables raw

QA and SA sit at 0.85–0.95 of optimal; by ~ 2300 variables raw QA frequently produces no budget-feasible sample in its best 20, and on the k-NN family SA is infeasible on all three smallest instances. This is the linear budget penalty doing what Proposition 3 says it does: λ trades off against the objective rather than enforcing the budget, and low-energy samples land on either side of B.

2. *Greedy repair is carrying the result.* On planar and Erdős–Rényi graphs the repaired solutions are optimal in all but two instances (the ER 1438(10) and 2370(12) rows, at 0.99 and 0.97), including every case where the raw sample was infeasible or far from optimal. QA-greedy and SA-greedy are indistinguishable everywhere. Slide 12 supplies the likely reason: on these random families the optimal interdiction set consisted of arcs incident to s or t, which a greedy "remove the highest-impact arc" pass finds without help. The experiment that separates the annealer's contribution from the repair's has not been run: greedy from a random or empty x, on the same instances.

3. *The k-NN family is the one with signal.* It is the only family where greedy repair fails to reach the optimum (0.90–0.95 on the six larger instances), so it is the only family in this deck on which a better sampler, a better penalty, or a warm start could visibly improve the reported number. It is also, not coincidentally, the family whose raw feasibility is worst.

4. *The QPU and SA are equivalent here.* No row separates QA from SA after repair, and raw QA is more often infeasible than raw SA at the largest sizes (consistent with embedding overhead on the Advantage topology at ~ 2300 logical variables). There is no quantum-advantage claim to be made from this data, and the deck does not make one.

The deck's motivating example (slides 1–2: a four-node network, remove two arcs, max flow $25 \to 5$, "an 80% reduction") is the same picture as Figure 1; as drawn on the slide, the three arcs leaving s have capacities $6 + 11 + 5 = 22$, so one capacity label there is stale relative to the stated 25 and is worth fixing before the deck is reused.

5 Existing work

Classical interdiction. Wood (1993), *Deterministic network interdiction*, Math. Comput. Modelling 17(2) — NP-hardness, the dualized formulation (2), and the linearization. Phillips (1993), *The network inhibition problem*, STOC — hardness in general, pseudo-polynomial algorithm for planar graphs. Israeli & Wood (2002), *Shortest-path network interdiction*, Networks 40(2) — Benders decomposition for interdiction, the template for keeping the flow subproblem separate. Cormican, Morton & Wood (1998), *Stochastic network interdiction*, Oper. Res. 46(2). Altner, Ergun & Uhan (2010), *The maximum flow network interdiction problem: valid inequalities, integrality gaps, and approximability*, Oper. Res. Lett. 38(1). Chestnut & Zenklusen (2017), *Hardness and approximation for network flow interdiction*, Networks 69(4). Smith & Song (2020), *A survey of network interdiction models and algorithms*, Eur. J. Oper. Res. 283(3) — the survey to start from.

QUBO / annealing machinery. Lucas (2014), *Ising formulations of many NP problems*, Front. Phys. 2 — the standard penalty encodings, including the squared cardinality penalty this repository does *not* use. Glover, Kochenberger & Du (2019), *Quantum bridge analytics I: a tutorial on formulating and using QUBO models*, 4OR 17. Boros & Hammer (2002), *Pseudo-Boolean optimization*, Discrete Appl. Math. 123, and Ishikawa (2011), *Transformation of general binary MRF*

minimization to the first-order case, IEEE TPAMI 33(6) — quadratization of higher-order terms, of which Proposition 1 is one instance. Hen & Spedalieri (2016), *Quantum annealing for constrained optimization*, Phys. Rev. Applied 5 — constraint-preserving (fixed Hamming weight) drivers, the annealing side of issue #3. Hadfield et al. (2019), *From the QAOA to a quantum alternating operator ansatz*, Algorithms 12(2) — the gate-model side of the same idea.

Warm starts. Egger, Mareček & Woerner (2021); He et al. / Shaydulin (2026); Cain et al. (2022); Tate et al. (2023) — detailed in Part II.

What is missing. I am not aware of a prior QUBO/annealing treatment of MFNIP specifically, and did not find one in the searches behind this note; that is a statement about the search, not the literature, and a proper check is a cheap first task. The repository's own QUBO construction — Lagrangian budget term, cubic dual penalty quadratized through w — has no written derivation anywhere; Section 4 is the first.

Part II — Exploratory research threads

Three threads follow. The first is the original brainstorm on whether SA/QA can use the convex relaxation the way branch-and-bound does; the second is what the warm-start literature actually prescribes; the third is the divide-and-conquer decomposition and subgradient penalty tuning marked work-in-progress in the slide deck. Section 9 then ties each thread back to the derivations in Part I and to the open issues.

6 Convex relaxations vs. SA/QA heuristics — the original brainstorm

The question that started this note: branch-and-bound gets its power from a convex relaxation; SA and QA don't use one — why not, and could they? A couple of the premises needed a small correction before the "why not" makes sense.

Is it purely discrete?

Yes — every decision variable in the repo's formulations (x, γ, π, and the auxiliary w) is binary. Edge weights are fixed data, not decisions. There's no continuous decision variable anywhere; this is a 0-1 program, not a mixed-integer program with a meaningful continuous part.

What B&B is actually doing (and a wrinkle worth knowing)

At each node, Gurobi solves the LP relaxation (integrality dropped, $x \in [0, 1]$) — that's convex, gives a valid bound, and branching + cutting planes progressively shrink the gap between that relaxation and the true integer hull. So yes, "convexify, bound, prune" is the right mental model.

One wrinkle: this problem's objective isn't linear to begin with. `nip_to_qubo` and `gurobi_verify_parameters` have bilinear terms $(1 - x)\gamma$ and, in the dual-linking term, a genuinely cubic term $\pi_u \pi_v \gamma$. For binary variables these are *exactly* linearizable (McCormick/RLT: a product of binaries is itself binary, so $z = a\,b\,c$ can be captured exactly, not just approximated, with a few linear constraints — that's what the `w` variable in `gurobi_verify_parameters` is doing). So what Gurobi is really solving is the LP relaxation of an exactly linearized reformulation, not a relaxation of the polynomial form directly.

The "strongly convex ⇒ local min = global min" intuition — where it applies and where it doesn't

That argument is about a continuous, differentiable objective. The LP relaxation here is convex but not strongly convex (it's linear), and its optimum being "the" optimum isn't because gradients vanish — it's because LP optima sit at polytope vertices and simplex/interior-point methods find the global one for free (convexity ⇒ no spurious local optima to get stuck at).

The catch: the LP relaxation's optimum is a *different problem's* optimum, not this problem's, and it only coincides with the true 0-1 optimum when the relaxation happens to be integral. Here's the interesting bit specific to this codebase: the *inner* max-flow/min-cut LP is integral automatically — network flow constraint matrices are totally unimodular, which is exactly why plain `nx.maximum_flow` never needs integrality constraints. The fractionality — and the NP-hardness — comes entirely from bolting the attacker's budget constraint ($\sum x \leq B$) and the duality-linking terms onto that otherwise-integral flow polytope. So the relaxation is tight everywhere except where the actual decision variables live, which is a pretty precise way of stating why this is hard.

So why can't SA/QA just use the convex relaxation the way B&B does?

Two separate reasons, and one of them dissolves into something more interesting than "can't."

1. Mechanically, SA and QA aren't gradient-following — they're stochastic search over discrete/spin configurations (bit-flips accepted/rejected via the Metropolis criterion, or quantum tunneling between basins). A relaxation's bound is only useful to something that does bounding-and-pruning (B&B); it doesn't plug into an update rule that has no notion of "prune this branch."

2. But — and this is the interesting part — annealing already does something structurally identical to what's being asked for, just with a generic surrogate instead of an LP-informed one. SA's high-temperature regime smooths the energy landscape into something close to convex (this is literally "deterministic annealing" / graduated non-convexity in the continuous-optimization literature — cooling is a homotopy from a smooth surface down to the true rugged one). QA does the quantum-mechanical version explicitly: it starts in the ground state of a trivial transverse-field Hamiltonian (unique ground state, no barriers — the "convex" end of the homotopy) and adiabatically deforms it into the problem Hamiltonian (the true nonconvex Ising landscape), tracking the ground state the whole way if the schedule is slow enough relative to the minimum spectral gap. That *is* "tie the convex surrogate's solution back to the nonconvex problem's

solution" — it's just using a problem-agnostic convex surrogate (equal superposition / infinite temperature) rather than one built from this problem's actual LP relaxation.

That reframes the question into a real, open one: **can the generic surrogate be replaced by one informed by the LP relaxation?** Concretely, three ways to actually do this, increasing in effort:

1. **Cheap classical baseline** — solve the LP relaxation once, round the fractional $x/\pi/\gamma$, then run the same greedy repair already sitting in `improve_dwave_solution` (currently only used to patch infeasible SA/QA samples) to project it onto the budget-feasible set. This costs one LP solve, no annealing at all, and gives a fast baseline to compare SA/QA against — right now there's no "how good is a dumb LP-round-and-repair" data point in the plan.
2. **Bound accounting for free** — solve the relaxation once, use its objective as a valid lower bound, and report the gap against whatever SA/QA/Gurobi returns. That's B&B's bounding step without the tree — a one-line addition that turns every SA/QA run into a run with a certified (or at least estimated) optimality gap instead of just a number.
3. **Warm-starting the annealer** — use the LP relaxation's fractional solution to bias the initial state (QA) or bias fields/initial temperature schedule (SA), instead of starting from the generic uniform/high-T surrogate. This is exactly what "warm-start QAOA" does in the literature, and it connects directly to the constraint-aware mixer issue (#3): a mixer restricted to the fixed-Hamming-weight (budget-respecting) subspace is a way of baking the polytope's structure into the search dynamics rather than penalizing it.

One more thing worth naming explicitly, since it falls straight out of the TU-network observation above: the classical OR literature on network interdiction doesn't usually solve the penalized single-level MILP the way this repo does — it typically uses **Benders decomposition**, exploiting exactly the fact that the inner max-flow LP is integral (Israeli & Wood's dualization is the same trick used here, but Benders keeps the flow/cut structure as a subproblem instead of flattening everything into one big linearized MILP with cubic terms). That's likely a stronger and faster classical baseline than what's currently in `network_interdiction.py`, and it would sharpen issue #1 too — it would tell us whether the "hard for Gurobi" instances are hard for the monolithic MILP specifically, or hard for network interdiction in general (Benders included).

7 Two concrete warm-start recipes from the literature — and whether we can implement them

Item 3 above waved at "warm-start QAOA." Two specific papers prescribe two different ways to build the warm start, and the difference matters for what we could actually run here. Throughout, Q is the QUBO matrix that `nip_to_qubo` already produces (as a `dimod` BQM), $n = 3E + V - 2$ is its variable count, and λ keeps its meaning from the rest of the repo (the cardinality penalty coefficient) — the regularization weight below is written μ to avoid the clash.

A. Egger, Mareček, Woerner — relaxation-seeded warm start (Quantum 5, 479, 2021)

Recipe. Solve a continuous relaxation of the QUBO once and read off $c^* \in [0,1]^n$. Prepare the product state

$$|\phi^*\rangle = \bigotimes_i \hat{R}_Y(\theta_i)\,|0\rangle, \qquad \theta_i = 2\arcsin\sqrt{c_i^*},$$

so that measuring qubit i gives 1 with probability exactly c_i^*. Then replace the standard mixer $-\sum_i \hat{X}_i$ by a rotated single-qubit mixer whose ground state is $|\phi^*\rangle$:

$$\hat{H}_{M,i}^{(\mathrm{ws})} = \begin{pmatrix} 2c_i^*-1 & -2\sqrt{c_i^*(1-c_i^*)} \\ -2\sqrt{c_i^*(1-c_i^*)} & 1-2c_i^* \end{pmatrix} = -\sin\theta_i\,\hat{X}_i - \cos\theta_i\,\hat{Z}_i,$$

which is implemented per layer as $\hat{R}_Y(\theta_i)\hat{R}_Z(-2\beta_k)\hat{R}_Y(-\theta_i)$ — no extra two-qubit gates over plain QAOA.

Why the regularizer. If the relaxation returns $c_i^* \in \{0,1\}$, qubit i starts in a $\hat{Z}$ eigenstate, and since the cost Hamiltonian is diagonal and the mixer is aligned with that state, nothing in the circuit ever moves it. That's a reachability failure whenever the relaxed and integer optima disagree on that bit. Their fix is to clip: pick $\varepsilon \in [0, 0.5]$ and set $\theta_i = 2\arcsin\sqrt{\varepsilon}$ when $c_i^* \le \varepsilon$, $2\arcsin\sqrt{1-\varepsilon}$ when $c_i^* \ge 1-\varepsilon$ (mixer adjusted to match). $\varepsilon = 0.5$ recovers standard QAOA exactly; their numerics use $\varepsilon = 0.25$. With $\varepsilon > 0$ the initial state overlaps every basis state, so the usual adiabatic argument for $p \to \infty$ convergence goes through.

What you get for free. At depth $p = 0$, measuring $|\phi^*\rangle$ *is* randomized rounding of the relaxation, so the algorithm inherits the classical guarantee (Goemans–Williamson's 0.878 for MaxCut when the relaxation is the SDP). Every extra layer can only be optimized on top of that.

Which relaxation. For MaxCut they use the GW SDP; for general QUBO the quadratic-program relaxation $\min_{x\in[0,1]^n} x^\top Q x$. That QP is convex only if $Q \succeq 0$, which ours is not — but on binaries $x_i^2 = x_i$, so adding $\sum_i d_i(x_i^2 - x_i)$ to the objective changes nothing on $\{0,1\}^n$ and any $d_i \ge \sum_{j\neq i} |Q_{ij}|$ makes the shifted matrix PSD (Gershgorin). The result is a box-constrained convex QP that `scipy` L-BFGS-B solves without Gurobi.

Mapping onto NIP. Two cautions specific to this repo:

- The obvious relaxation to reach for is the LP relaxation of the linearized MILP (the one with the w variables in `gurobi_verify_parameters`). That is *not* what `solve_nip_gurobi_relaxation` computes — that function is a Lagrangian bisection on λ with the variables still binary, so it returns an integer x, i.e. a warm *bitstring*. Seeding QAOA with a bitstring and the standard mixer is precisely the setting Cain et al. show gets stuck (arXiv:2207.05089): a good guess is a near-eigenstate of the cost, and the $\hat{X}$ mixer then makes almost no progress. So the Lagrangian output is usable only through the ε-clipped state and rotated mixer, never as a bare initial state.
- By the TU observation earlier, the $\pi/\gamma/w$ block is determined by x through a min-cut, so a relaxation is fractional essentially only on the x coordinates. That is where ε does its work; the rest of the qubits will sit near $\{0,1\}$ and be clipped to ε regardless.

B. He, Apte, Augustino, Babakhani, Khan, Omanakuttan, Shaydulin — regularized warm start (arXiv:2603.10191, 2026)

Recipe. No relaxation solve at all. Choose the product state directly by optimizing its Bernoulli parameters $p_i = \sin^2(\theta_i/2)$ against the *mean-field energy plus a regularizer*:

$$\min_{p\in[0,1]^n} \mathcal{L}_\mu(p) = \mathbb{E}_{x\sim p}\big[x^\top Q x\big] - 4\mu \sum_i p_i(1-p_i), \qquad \mathbb{E}_{x\sim p}\big[x^\top Q x\big] = \sum_i Q_{ii}\, p_i + \sum_{i<j} Q_{ij}\, p_i p_j.$$

The expectation is closed-form under a product distribution (each term costs one nonzero of Q), so a gradient step is $O(\mathrm{nnz}(Q))$ and the whole thing is cheap at any size we care about. The regularizer is maximal at $p_i = \frac{1}{2}$ and zero at a bitstring, so subtracting it pulls every qubit *away* from the classical extremes: $\mu \to \infty$ gives $p \equiv \frac{1}{2}$ (standard QAOA), $\mu \to 0$ gives a mean-field collapse onto a bitstring. The mixer is the same rotated family as A — $\hat{H}_M(\theta) = \sum_i \sin\theta_i\, \hat{X}_i + \cos\theta_i\, \hat{Z}_i$ up to sign convention — aligned so the optimized state is its ground state. They run it *non-variationally*: (γ, β) are fixed in advance by optimizing on local subgraphs of representative instances and then transferred, which is what lets them go to $N = 10{,}000$ at depth 6 with tensor networks and to 96 nodes on Quantinuum hardware.

Why it doesn't stall. Their argument is the clean version of the reachability problem in A: if the initial product state is a bitstring, it is a common eigenstate of the cost Hamiltonian and of the aligned mixer, so the QAOA unitaries act on it only by a phase and the algorithm explores nothing. The regularizer makes that state unreachable by construction rather than by post-hoc clipping. For D-regular MaxCut they show $\mu > D/4$ makes $\mathcal{L}_\mu$ strictly convex with a unique minimizer (they nonetheless find $\mu = 0.6$ works best for 3-regular, below the guarantee). For our weighted, non-regular Q the same Gershgorin argument gives the sufficient condition $\mu > \frac{1}{8}\max_i \sum_{j\neq i} |Q_{ij}|$ with Q_{ij} the BQM's quadratic biases counted once per pair — large for us, because the penalty terms with coefficient λ sit in exactly those off-diagonals. Expect the useful μ to be well below that bound, as it was for them.

Scope. Stated for any QUBO; the paper explicitly leaves constrained problems as future work. Our penalized QUBO qualifies as a QUBO, but note the regularizer and the cardinality penalty now compete inside $\mathcal{L}_\mu$: λ wants the mean-field x to have $\sum p_i \approx B$, μ wants every p_i near $\frac{1}{2}$. On an instance with $E \gg B$ those pull in opposite directions, so the μ window that neither collapses nor washes out the budget is another thing to sweep, on top of the λ window issue #2 is already about.

C. What is implementable now, with what is installed

1. **The RWS product-state optimizer, today.** $\mathcal{L}_\mu$ and its gradient are a few dozen lines of `numpy` over the `dimod` BQM `nip_to_qubo` already returns; projected gradient descent on $[0,1]^n$. No Gurobi, no LP, no new dependency. Output: $p \in [0,1]^n$.

2. **Using p without any gate-model machinery.** The annealer analogue of "start in $|\phi^*\rangle$" is "start SA from bitstrings drawn from p": `neal.SimulatedAnnealingSampler.sample` accepts `initial_states`, and `solve_qubo_via_dwave.py` already uses that argument for the steepest-descent pass. Sampling 2500 initial states from Bernoulli(p) instead of uniformly is a one-line change in the `sa` path, keeps the regularizer's diversity (it does not collapse every read onto one guess), and is directly measurable against the sample benchmark by adding an `sa-ws` pass to `run_benchmark.py`. On the QPU the analogue is reverse annealing (`initial_state`

+ `anneal_schedule` + `reinitialize_state` on `DWaveSampler`) from a rounded p. Neither is QAOA, but both are the same idea on the stack we have, and either would tell us within an afternoon whether relaxation-informed starts move the SA/QA numbers at all.

3. **Actual WS-QAOA / RWS-QAOA needs a gate-model simulator**, which is not in `requirements.txt`. `qiskit-optimization` ships Egger's method as `WarmStartQAOAOptimizer` (with the ε knob); RWS would be a custom initial state plus the rotated mixer on top of any QAOA implementation. Statevector simulation covers the 14–22-qubit `tiny_*` instances comfortably and nothing near `50.planarflow` ($n \approx 300$), so this is a tool for the mixer study in #3, not for the scaling question in #1. It is a dependency decision worth making deliberately.

4. **Egger's route additionally needs the relaxation.** The convexified QP above is the cheapest honest option (scipy, no license). The LP relaxation of the linearized MILP is the one with the OR-textbook meaning, but it needs an LP solver — Gurobi when licensed, otherwise `scipy.optimize.linprog` (HiGHS) on a model we would have to build by hand since `gurobi_verify_parameters` builds it in `gurobipy` terms. Worth noting that these two relaxations are genuinely different objects: the QP relaxes the *penalized* QUBO, so its fractional c^* is partly shaped by λ rather than by flow structure, whereas the MILP LP relaxation keeps the budget as a constraint. Comparing the warm starts they produce would itself be informative.

5. **The open design question is composition with constraint-preserving mixers.** Both warm-start mixers are products of single-qubit rotations and do not preserve Hamming weight; the fixed-$\sum x = B$ mixers from #3 do, and are not products. Neither paper addresses combining the two (RWS names constrained problems as future work; Tate et al.'s custom-mixer line, arXiv:2112.11354, is also unconstrained). A warm start inside a constraint-preserving subspace would need the initial state to live in that subspace — a Dicke-like state biased by p, not a product state — and a mixer aligned to it. That is a research item, not an implementation item, and it is the one place where this thread and #3 actually meet rather than merely sit next to each other.

Sources. Egger, Mareček, Woerner, "Warm-starting quantum optimization," Quantum **5**, 479 (2021), arXiv:2009.10095. He, Apte, Augustino, Babakhani, Khan, Omanakuttan, Shaydulin, "Regularized warm-started quantum approximate optimization and conditions for surpassing classical solvers on the Max-Cut problem," arXiv:2603.10191 (2026). Cain, Farhi, Gutmann, Ranard, Tang, "The QAOA gets stuck starting from a good guess," arXiv:2207.05089. Tate, Moondra, Gard, Mohler, Gupta, "Warm-started QAOA with custom mixers provably converges and computationally beats Goemans–Williamson's Max-Cut at low circuit depths," arXiv:2112.11354.

8 Divide and conquer, and subgradient tuning of λ — work in progress from the slides

Slides 11–12 of `S-T_NIP.pptx` sketch two further directions. Both are recorded here as stated, with what Part I adds to each.

Decomposition

As proposed. Partition the unweighted graph into contiguous pieces (METIS, or an edge separator); assign each arc to the piece of its tail, so separator arcs are owned by the piece their source lies in. Split the budget B between pieces either in proportion to how many arcs of a global min cut each piece owns, or by estimating each piece's local max flow with super-source/super-sink nodes attached to its boundary vertices. Solve an NIP in each piece and combine. The current prototype keeps the whole graph and only restricts which arcs may be interdicted, which shrinks the classical search space but leaves the QUBO the same size — the slide's own criticism — and it floats contracting the other piece to a single node when solving one.

What Part I adds.

- The budget split is the entire difficulty, and it is a Lagrangian question. If piece P gets budget B_P, the combined solution is optimal only if the marginal value of the last arc in each piece is equalized — which is exactly the statement that one multiplier λ serves all pieces. So a split by ratios of cut arcs or local flows is a heuristic for a common λ, and by Proposition 3 a common λ exists only if each piece's $f_P(k)$ is convex at its B_P. The decomposition and the penalty question are the same question.
- Interdicting arcs in one piece changes the min cut and therefore the boundary flows of the other; local max flows computed once are stale after the first interdiction. Benders decomposition (Part I, existing work) is the standard remedy: keep the pieces as flow subproblems and iterate cuts, rather than fixing the split up front.
- Contraction is sound only if the contracted piece is not interdicted — otherwise its internal capacity is not a constant. A contracted piece with budget 0 can be replaced by its local max flow exactly; that is the case where contraction is a proof, not a heuristic.

Subgradient tuning of the cardinality penalty

As proposed. Drop the β term (it is zero at any optimum, by Proposition 2), write $\mathcal{L}(x,\lambda) = \sum_a u_a(1-x_a)\gamma_a + \lambda(\sum_a x_a - B)$, and maximize the dual $d(\lambda) = \inf_x \mathcal{L}(x,\lambda)$ by subgradient ascent: the subgradient at λ is $\sum_a x_a^\lambda - B$, the excess of interdicted arcs over budget, where x^λ minimizes $\mathcal{L}(\cdot,\lambda)$. The slide's stated problem: the minimizer x^λ has to come from a heuristic QUBO solver.

What Part I adds.

- $d(\lambda) = \min_k f(k) + \lambda(k-B)$ is the lower convex envelope of f evaluated at slope $-\lambda$, so $\max_\lambda d(\lambda)$ is the tightest Lagrangian bound and is attained on the window (6) when that window is non-empty, and strictly below $f(B)$ otherwise. Subgradient ascent on d is the principled version of the bisection on slide 7: same object, with a convergence theory and a certificate ($d(\lambda)$ is a valid lower bound at every iterate, heuristic or not).
- The heuristic-argmin worry is real but asymmetric. A heuristic $\tilde{x}$ gives $\mathcal{L}(\tilde{x},\lambda) \geq d(\lambda)$, so it *overestimates* the bound (unsafe as a certificate) but its excess $\sum_a \tilde{x}_a - B$ is still a usable ascent direction in expectation. Using the exact decode (the max-flow evaluation the pipeline already does) rather than the QUBO energy for $\mathcal{L}(\tilde{x},\lambda)$ removes the β and offset noise from that estimate.

- On the six sample rows, d is computable exactly (Table 1 lists f), which makes them the right unit test for a subgradient implementation before it is pointed at an annealer.

9 Tying the threads to the results

Each exploratory idea above rests on something specific in Part I. Making that dependence explicit is what turns a brainstorm into a work plan, because it says what has to be true — or measured — before the idea is worth an implementation.

The penalty structure comes first (issue #2)

Every heuristic result in this project is measured against the ground state of (5), and Propositions 2–3 say that ground state is the interdiction optimum only inside a (λ, β) region that is instance-dependent, sometimes narrow, and sometimes empty. Three consequences for the rest of the plan:

- **The λ window is computable, not searchable.** By (6) it needs $f(k)$ for k near B — on tiny instances by enumeration, on real ones by a handful of exact solves at neighbouring budgets, or by bounds on f (the LP relaxation gives a lower bound on $f(k)$, the greedy repair an upper bound, and the window only needs the difference quotients). The current practice of sweeping λ with an exact solver in the loop is doing by search what (6) does in closed form. This is the concrete deliverable for #2, and Table 1 is already its test.
- **When the window is empty, no λ can be "derived."** That is the case where f is not convex at B. The fix is structural, not numerical: replace the linear term by the squared penalty $\lambda(\sum_a x_a - B)^2$ (Lucas 2014), whose valid range is a one-sided bound and always non-empty, at the cost of $O(|A|^2)$ new quadratic couplings; or keep the linear term and accept that the QUBO solves the Lagrangian dual, reporting $L(\lambda)$ as a bound rather than the answer. Which of these the project wants is a decision, and it should be made with the convexity of f on the planar instances measured first.
- **$\beta = \lambda + 1$ is already right; #2 is about λ alone.** Proposition 2 confirms the slide-deck argument that any $\beta > \lambda$ makes the ground state violation-free, independent of capacities, so the dual penalty needs no instance analysis. What does need one is the *typical* width of the λ window (6) on the benchmark families — how often it is empty, and how narrow it is when it is not — since that is what decides whether a fixed or bisected λ can work at all. The deterministic / almost-sure / average-case split used for the balanced-bisection penalty threshold in the sibling preconditioning project is the right template for that: worst case over capacities, the threshold under random capacities, and the per-instance window width in the bulk.

The relaxation threads (Part II, first thread) each rest on a different fact

- *LP-round-and-repair baseline* needs the LP relaxation of (2), which the repository does not have (its "relaxation" function is the Lagrangian (3)). Building it is one `scipy.optimize.linprog` model. Altner et al.'s integrality-gap result predicts that the rounded LP solution will often be

poor — which makes the baseline *more* informative, not less: if SA/QA only match LP-round-and-repair, they are not extracting anything the LP did not already know.

- *Gap certificates* come from the same LP, or for free from $L(\lambda)$ in (3): by weak duality every Lagrangian value is a lower bound on the interdiction optimum, so the bisection the repository already runs produces certificates it currently throws away.
- *Benders decomposition* is the classical baseline that respects the TU structure (the cut subproblem is a max-flow, never a MILP). Its relevance to issue #1 is the same as Phillips' planar result: both say the planar instances may be hard for the monolithic formulation and easy in fact. The benchmark set should be validated against Benders before anything is declared hard.

The warm-start thread (Part II, second thread) depends on the penalty structure twice

- Both warm starts operate on the QUBO (5), so both inherit the (λ, β) question: a warm start toward the ground state of a QUBO whose ground state is wrong is a warm start toward the wrong answer. Fix #2 first, or run the warm-start experiments only on rows inside the window of Table 1.
- The regularized (RWS) objective $\mathcal{L}_\mu$ contains the Lagrangian term $\lambda \sum_a p_a$ through $\mathbb{E}[x^\top Q x]$, so λ and μ interact: λ pushes the mean-field $\sum_a p_a$ down toward B, μ pushes every p_a toward $\frac{1}{2}$, and with $|A| \gg B$ those are opposed. The window of useful μ is therefore a function of λ, and the sufficient uniqueness bound $\mu > \frac{1}{8} \max_i \sum_j |Q_{ij}|$ scales with β through the dual-penalty couplings. This is a two-parameter sweep on the tiny rows before it is anything else.
- The cheapest experiment in the whole document is the one that needs none of the gate-model machinery: RWS product state → Bernoulli(p) initial states → `neal` with `initial_states`. It tests the *hypothesis* behind warm starts (that a relaxation-shaped start beats a uniform one on this landscape) on the solver the project already benchmarks, and it slots into `run_benchmark.py` as one more pass. If it does nothing on the planar instances, the gate-model version is unlikely to either, and the project learns that for the price of an afternoon.
- The composition of warm starts with constraint-preserving mixers (#3) is where this thread and the penalty thread finally meet: a fixed-Hamming-weight mixer removes the need for the λ term altogether (the budget is enforced by the subspace, so (6) becomes moot), leaving only β. That is a strong argument for #3 on its own, independent of warm starts, and it is the version of the problem in which a warm start would need to be a biased Dicke-type state rather than a product state — an open construction.

The D-Wave results (Section 4.1) change what the benchmark question is

- Table 2 says the reported optimality comes from greedy repair on two of three families, and slide 12 says why (optimal arcs incident to s/t). Before any instance is called hard for issue #1, run greedy repair from an empty and from a random x on the same instances; whatever it reaches is the floor any annealer result has to beat. The deck's own next step — real flow networks from the DIMACS max-flow collections instead of random families — is the right one, and Phillips' planar result (Part I) says the planar family should be dropped from the "hard" set regardless.

- The k-NN family is where a method change could show: it is the only family with headroom after repair. It should be the primary target for the warm-start pass and for any change to the budget penalty.
- Raw infeasibility at ~ 2300 variables is the linear-penalty tradeoff of Proposition 3 made visible. The squared penalty would fix feasibility at the cost of density; a fixed-Hamming-weight mixer (#3) would fix it at no density cost but only on gate-model hardware. This is the concrete evidence the linear-vs-squared decision in #2 should be made on.

The decomposition thread (Section 8) is downstream of #2, not parallel to it

The budget split between pieces is a common-multiplier problem, so it inherits every property of the λ window; and the subgradient method is the principled form of the bisection already in the code, with the sample rows as its exact unit test. The order is therefore: subgradient tuning of λ on whole instances first (it is small, certifiable, and slots into `run_benchmark.py`), decomposition after, and contraction only for zero-budget pieces where it is exact.

Ordering

The dependencies above give an order that is not the order the issues were filed in (issue numbers as of Section 10):

1. Recover the CSVs behind slides 8–10 into `results/` so Table 2 stops being read off a figure, and add the greedy-from-scratch baseline on the same instances (#18, #15).
2. Measure $f(k)$ near B on the benchmark instances and compute the λ window from (6); implement subgradient tuning of λ with the sample rows as its unit test; decide linear vs. squared budget penalty on the raw-feasibility evidence in Table 2 (#2).
3. Add the LP relaxation and its two free by-products (LP-round-repair baseline, gap certificate), and a Benders baseline (#15); replace the random families with real networks from the proposal's application domains and re-examine which instances are hard for what (#1).
4. Run the `sa-ws` regularized-warm-start pass, on the k-NN family first; only if it moves the numbers, take on a gate-model QAOA dependency for the mixer study (#3), which is also the prerequisite for parameter transfer (#16) and the IBM path (#17).
5. Decomposition (#14) last, framed as a common-multiplier problem, with contraction restricted to zero-budget pieces; restore the QCI path (#17) whenever the client code is located, since it is independent of the rest.

10 The CQT Year-4 proposal: commitments and the issues that carry them

The proposal *Decomposition-Based Approaches for Practical Quantum Optimization* (Bernal, Pothen; CQT Year 4) makes four commitments that touch this repository. Table 3 maps each

to the issue that now tracks it; the ones numbered 14–18 were opened from this note, the comments on #1 and #2 were posted at the same time, and everything else is pre-existing.

proposal text	where it lives
Project description: NIP formulated as a QUBO by dualize-and-combine then Lagrangian relaxation; "only a small, dense submatrix"; on D-Wave with synthetic networks "over 80% optimal in small-to-mid instances, and greedy post-processing recovers optimal solutions in all instances."	Formulation: Part I, §2–§4 (the linear Lagrangian term is what keeps the QUBO sparse; Prop. 3 is its price). Claim: **#18** — Table 2 shows it holds for the planar family only; ER has two sub-optimal rows, k-NN six at 0.90–0.95; raw feasibility fails at $\sim$ 2300 variables. The CSVs behind the figures are to be recovered into `results/`.
Milestone Q1: "further increase the sizes of network interdiction problems that can be solved on NISQ devices via decomposition-based refinement."	**#14** (new), built on the slide-deck design (§8) with the common-multiplier analysis; #4 keeps the scaling-axis inventory.
Milestone Q1: "apply transfer parameter learning to increase scalability" (the GAT parameter-transfer approach from the MIS work, proposal ref. [5]).	**#16** (new), which depends on a QAOA existing first (#3) and notes that RWS-QAOA (§7) is the same idea with a regularized warm start; #2 because transfer across λ regimes transfers to the wrong Hamiltonian.
Milestone Q3–4: "comparing decomposition approaches against classical methods."	**#15** (new): greedy-from-scratch, LP round-and-repair with its gap certificate, Lagrangian bound accounting, and Benders — the four baselines Part II's first thread and the tie-in called for; none existed as an issue.
Milestone Q3–4: "test implementations on D-Wave, QCI, and IBM devices."	**#17** (new): the `qci` branch calls a `solve_qubo_via_qci` module that is not in the repository (`NameError` if reached) and expects the Julia-era file format; there is no gate-model path for IBM at all until #3/#16 produce one.
Industry/government need: adversary interception, infectious disease control, power-system failure isolation.	Comment on **#1**: one real network per named domain; exclude instances whose optimal interdiction is incident to s/t (slide 12); time Benders as well as the MILP before calling anything hard; drop the planar family from the hard set (Phillips 1993).
— (not in the proposal, but the derivations in §4 change the task)	Comment on **#2**: $\beta > \lambda$ suffices (no tuning); λ has the exact two-sided window (6), sometimes empty, so the deliverable is the window computation, the linear-vs-squared decision, and subgradient tuning — not a one-sided lower bound.

Table 3: Proposal commitments that concern the s–t interdiction problem, and the issue or note section that now carries each. Issues #14–#18 were opened from this note on 2026-09-16.

Two things in the proposal text deserve a flag beyond the table. The results sentence is stronger than the data in the group's own slides, and it will be read by the site directors against whatever the Year-4 talk or poster shows; #18 exists so that the next version of that sentence is by family. And the proposal's second milestone — integer encodings for QUIO — is not an s–t interdiction item and has no issue here on purpose; if the budget variable or the capacities are ever encoded as integers rather than penalized binaries, that is where it would connect.

Open question: open issues for these — (1) LP-round-repair baseline, (2) relaxation-based gap certificates for SA/QA/QAOA runs, (3) LP-informed warm-starting tied into the mixer work, (4) Benders decomposition as a stronger classical baseline, (5) a regularized-warm-start `sa-ws` pass in `run_benchmark.py` as the cheap first experiment, (6) computing the λ window from (6) and subgradient tuning of λ as the deliverable for #2, (7) the greedy-from-scratch baseline and real-network instances that Table 2 calls for, (8) decomposition as a common-multiplier problem — (1)–(4) and (7)–(8) are now #15, #14, and the comments on #1/#2; (5)–(6) remain to be filed — in the order of Section 9, or keep brainstorming first?